



**Universidad
de La Laguna**

Práctica 4: Programación Dinámica

Por

Javier Acosta Portocarrero - alu0101660769

**Diseño y Análisis de Algoritmos
Escuela Superior de Ingeniería y Tecnología**

Índice

Índice	2
Estructura	3
Contenido de la clase AlgoritmoDinámicoPlanificaciónDía	5
AlgoritmoDinámicoPlanificaciónDía::EncontrarCantidadSlots	5
AlgoritmoDinámicoPlanificaciónDía::TraducirSlotTurno	5
AlgoritmoDinámicoPlanificaciónDía::ConstruirTablaDinamica	5
AlgoritmoDinámicoPlanificaciónDía::ReconstuirSolucion	6
AlgoritmoDinámicoPlanificaciónDía::Solve	6
Experimentación	7
Tiempos de ejecución	7
Calidad de las soluciones	9
Conclusiones	10
Extensión	11

Estructura

He partido de la estructura usada en el programa de la práctica 4, que es la base del realizado para esta práctica, la cual se ha tratado en profundidad en el [anterior informe](#).

Sobre esta estructura usada como base, cabe destacar que la clase [AlgoritmoAproximadoPlanificacion](#), que hereda de la clase abstracta plantilla de algoritmos Divide y Vencerás, ya hacía uso de un puntero a la clase abstracta general de algoritmos, usado para resolver las instancias suficientemente pequeñas, característica que ha sido usada para facilitar el intercambio de estrategias (tipos de algoritmos) en esta práctica.

Además, la clase [AlgoritmoAproximadoPlanificacion](#) ha sido modificada para permitir especificar a través del constructor cual debe ser el tamaño de una instancia para ser considerada “pequeña” y en cuántas subinstancias se realizan las divisiones en el método *Divide*. Esto será de especial importancia más adelante.

A continuación se tratarán las principales nuevas clases implementadas:

- [AlgoritmoDinamicoPlanificacionDia](#): Esta clase, que hereda de la clase Algoritmo ([Algoritmo](#)), reescribe su método Solve para resolver la instancia cuando es de tamaño 1. Este algoritmo sigue un enfoque de programación dinámica, es decir, para resolver las instancias, construye una tabla usando la ecuación recurrente especificada en el enunciado de la práctica, la cual usará para reconstruir la solución global de las mismas gracias a las soluciones de problemas más pequeños halladas. Más adelante se tratará en más profundidad el contenido de esta clase.
- [FactoryAlgoritmosPlanificacion](#): Esta clase abstracta define la interfaz común para todas las fábricas de algoritmos de planificación. Su objetivo es proporcionar un método virtual Crear(const json&) que permita generar objetos de la clase abstracta Algoritmo a partir de la configuración especificada en un objeto JSON. De esta forma se establece una jerarquía de fábricas concretas que implementan la creación de los distintos algoritmos disponibles, siguiendo el patrón de diseño **Factory Method**. Este diseño permite desacoplar la creación de los algoritmos de su utilización, favoreciendo además la extensibilidad del sistema y siguiendo los **principios SOLID**.

- [FactoryAlgoritmoVoraz y FactoryAlgoritmoDinamico](#): Estas clases heredan de la clase abstracta *FactoryAlgoritmosPlanificacion* y se encargan de la creación de los algoritmos concretos de tipo voraz y de programación dinámica respectivamente. Cada una implementa el método *Crear(const json&)*, el cual devuelve una instancia del algoritmo correspondiente. En estos casos el contenido del JSON únicamente determina el tipo de algoritmo a utilizar, por lo que la creación del objeto es directa.
- [FactoryDivideYVencerás y FactoryDivideYVencerásBinario](#): Estas clases también heredan de *FactoryAlgoritmosPlanificacion* y permiten la creación de algoritmos de tipo Divide y Vencerás. A partir de la información contenida en el JSON, estas fábricas configuran los parámetros necesarios del algoritmo, tales como el tamaño de la instancia pequeña (*smallSize*), el algoritmo encargado de resolver los casos base (*smallSolver*) y, en el caso del Divide y Vencerás general, el número de subinstancias generadas durante la fase de división (*divisions*). Para crear el algoritmo encargado de resolver los casos pequeños, estas clases utilizan el selector de fábricas, permitiendo así construir de forma recursiva configuraciones de algoritmos más complejas.
- [SelectorFactoryAlgoritmo](#): Esta clase actúa como componente encargado de seleccionar la fábrica concreta que debe utilizarse para crear un algoritmo determinado. A partir del campo "*class*" especificado en el JSON, el método *Seleccionar* devuelve una instancia de la fábrica correspondiente (voraz, programación dinámica o divide y vencerás). Este mecanismo permite desacoplar completamente el proceso de selección de la fábrica del resto del sistema, facilitando la incorporación de nuevos algoritmos sin necesidad de modificar el código existente, lo que contribuye al cumplimiento del principio **Open/Closed**. Además, este diseño permite combinar el patrón **Factory Method** con el **patrón Estrategia**, ya que el algoritmo creado se utiliza posteriormente como estrategia dentro de la clase encargada de realizar la planificación de empleados.
- [GeneradorInstanciasPlanificacion](#): Esta clase es usada para comparar los distintos algoritmos implementados. Contiene la implementación del método *GenerarInstancia(...)*, el cual recibe 3 parámetros numéricos, que se corresponden con las cantidades de empleados, días y turnos de la instancia a generar respectivamente, y genera una instancia con estas características, con valores de satisfacción, mínimo de empleados por turnos y descansos por empleado generados aleatoriamente.

Contenido de la clase

AlgoritmoDinámicoPlanificaciónDía

A continuación se tratarán los distintos métodos de la clase, indicando el razonamiento seguido, pero si se desea profundizar más, en el código al que llevan los enlaces adjuntos anteriormente hay comentarios más exhaustivos de todos los procedimientos llevados a cabo, sus motivos y el por qué del orden en el que se realizan.

Para la implementación de esta clase se han seguido las instrucciones indicadas en el enunciado de la práctica, donde se especificaba cómo construir la tabla del algoritmo de programación dinámica, la cual está basada en slots.

AlgoritmoDinámicoPlanificaciónDía::EncontrarCantidadSlots

Este método es el encargado de, a partir de una instancia, calcular la cantidad de slots (que serán usados como columna en la tabla de programación dinámica) que tiene. Los slots son la cantidad de empleados mínimos que tienen en total todos los turnos de la instancia.

AlgoritmoDinámicoPlanificaciónDía::TraducirSlotTurno

Este método traduce, para un objeto *SolucionPlanificacionEmpleados* recibido por parámetro, a qué turno exactamente pertenece un slot, el cual es pasado también por parámetro. Esto es será necesario más adelante para reconstruir la solución obtenida a través del algoritmo dinámico a partir de la tabla construida.

AlgoritmoDinámicoPlanificaciónDía::ConstruirTablaDinamica

Este método es el encargado de construir la tabla dinámica, conteniendo la lógica principal del algoritmo. Para construir la tabla usa una matriz, donde las filas se corresponden con los empleados y las columnas con los slots ([empleador][slot]), que se va recorriendo y, para cada celda, se elige la satisfacción con el siguiente criterio:

```
tabla_dinamica[empleados][slot] = std::max(  
    tabla_dinamica[empleados - 1][slot],  
    tabla_dinamica[empleados - 1][slot - 1] + satisfaccion_empleado);
```

Además, tiene en cuenta los casos base, que son en los que no hay ningún slot o ningún empleado (empleado = 0 o slot = 0), y hace uso del método encargado de traducir slots en turnos, para así hallar la satisfacción real de los empleados en el slot (y también hace uso del método que halla la cantidad de slots).

AlgoritmoDinámicoPlanificaciónDía::ReconstuirSolucion

Este método es el encargado de, a partir de la tabla dinámica ya construida, reconstruir cuál es la solución (asignación de empleados a cada turno) que ha llevado a la misma.

Para lograr esto comienza a recorrer la tabla de forma inversa, es decir, comienza desde la última celda (empleados y slots completos) y va detectando, para cada columna (slot), qué empleado se le ha asignado, si es que se le ha asignado alguno. Para poder realizar esto y asignar cada empleado a su correspondiente turno, hace uso del método que traduce slots en turnos concretos.

Procedimiento seguido:

```
size_t cantidad_empleados_actual = tabla.size() - 1;
size_t slot_actual = tabla[0].size() - 1;
while (cantidad_empleados_actual > 0 && slot_actual > 0) {
    if (tabla[cantidad_empleados_actual][slot_actual] != tabla[cantidad_empleados_actual - 1][slot_actual]) {
        unsigned turno_slot = TraducirSlotTurno(slot_actual - 1, solucion);
        solucion->NuevoTrabajoTurno(0, turno_slot, cantidad_empleados_actual - 1);
        --slot_actual;
    }
    --cantidad_empleados_actual;
}
```

AlgoritmoDinámicoPlanificaciónDía::Solve

Este método, que es la sobrecarga del método *Solve* heredado de la clase *Algoritmo* (siguiendo el **patrón estrategia** y los **principios SOLID**), es el encargado de, a partir de una instancia de un único día, hallar su solución siguiendo un enfoque de programación dinámica. Para hacer esto simplemente hace uso del método que construye la tabla dinámica, para después reconstruir la solución haciendo uso del método correspondiente.

Experimentación

Para realizar la experimentación, he usado la clase generadora de instancias aleatorias, modificando la cantidad de días de estas de forma incremental para poder ver cómo se comportan los dos algoritmos implementados, es decir, tanto el voraz como el de programación dinámica.

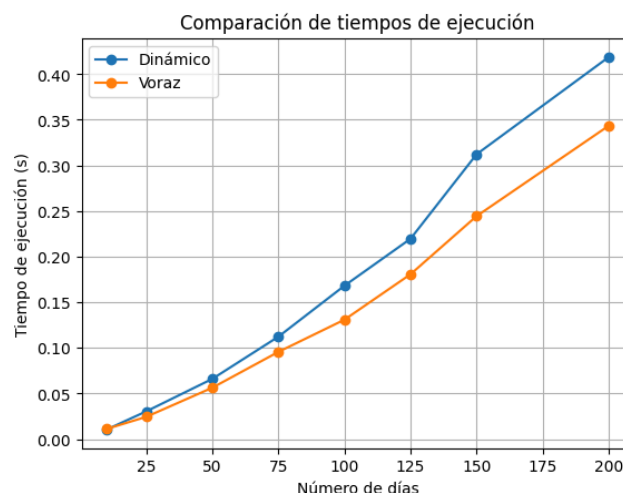
La variación de la cantidad de turnos y de empleados también influye, pero realizando pruebas me di cuenta de qué eligiendo buenos valores de estos parámetros, era suficiente con variar la cantidad de días para estudiar el comportamiento de los algoritmos, pues los resultados de variar las otras dos dimensiones eran muy similares. Además, hacer esto es necesario para visualizar el comportamiento de los algoritmos de forma clara, pues si se tuvieran en cuenta más variables, no podría generar gráficas precisas en dos dimensiones para ilustrar los resultados.

En la experimentación se han tenido en cuenta dos aspectos, la calidad de las soluciones (valores objetivos) y el tiempo de ejecución.

Para conseguir resultados más estables, para cada dimensión de instancia se ha ejecutado 5 veces cada algoritmo, para después poder calcular la media de los resultados obtenidos y obtener unas mediciones más precisas.

Tiempos de ejecución

Para poder visualizar de forma clara los resultados obtenidos, haré uso de una gráfica que he generado con el siguiente [cuaderno de Google Collab](#), en la que podemos ver cómo evolucionan los tiempos de ejecución para cada uno de los algoritmos:



Como se puede apreciar, el algoritmo dinámico es notablemente más lento para todo tamaño de instancias, lo cual se evidencia más en cuanto crecen las dimensiones, aunque no de forma exponencial, es decir, la diferencia no crece de forma exagerada.

Esto se debe a que el algoritmo dinámico construye una tabla de programación dinámica en la que se evalúan diferentes combinaciones posibles entre empleados y slots de turnos. Para ello, se recorren todos los empleados y todos los slots disponibles, calculando en cada posición de la tabla el valor óptimo entre no asignar al empleado o asignarlo al turno correspondiente. Este proceso implica rellenar una estructura bidimensional cuyo tamaño depende del número de empleados y del número de slots, por lo que el número de operaciones necesarias es mayor.

Además, el algoritmo debe realizar operaciones adicionales que también incrementan el tiempo de ejecución. Por un lado, es necesario traducir cada slot al turno correspondiente, ya que los slots representan únicamente las posiciones mínimas que deben cubrirse en cada turno. Para ello se utiliza una función que recorre los turnos acumulando el número mínimo de empleados hasta encontrar a cuál pertenece cada slot.

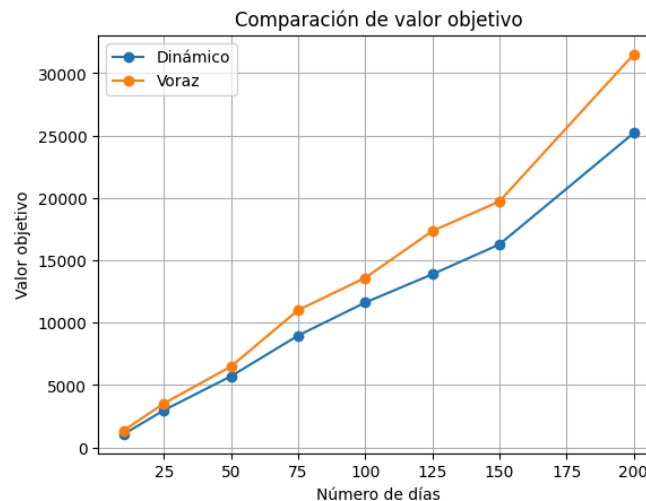
Por otro lado, una vez construida la tabla dinámica, es necesario reconstruir la solución final, lo cual también conlleva una complejidad adicional.

Por el contrario, el algoritmo voraz sigue una estrategia mucho más simple: en cada paso selecciona el mejor empleado disponible para un turno concreto. Esta decisión se toma de forma local sin considerar todas las combinaciones posibles, lo que reduce significativamente el número de cálculos necesarios.

En consecuencia, el algoritmo voraz presenta menores tiempos de ejecución, ya que evita construir estructuras auxiliares grandes y no evalúa todas las posibles asignaciones.

Calidad de las soluciones

Para medir la calidad de las soluciones haré uso de los valores objetivos obtenidos, y, al igual que para los tiempos de ejecución, haré uso de la gráfica generada a partir de los resultados obtenidos, esta gráfica también ha sido generada usando el mismo [cuaderno de Google Collab](#):



Como se puede apreciar, el algoritmo dinámico proporciona peores soluciones para todas las instancias probadas, lo cual, al igual que antes, se evidencia más en cuanto crecen las dimensiones, aunque no de forma exagerada.

Esto se debe principalmente a que el algoritmo dinámico solo asigna la cantidad de empleados mínima cada turno, mientras que el algoritmo voraz también asigna empleados extras a cada turno. Esto se traduce en una menor satisfacción local en la instancia de un solo día, y en que sea mucho más fácil romper la cantidad mínima de empleados mínimos en los turnos al hacer las combinaciones de soluciones y reparar la restricción de los descansos de empleados, ya que al no haber empleados extras en los turnos, con eliminar un único empleado ya se rompe esta condición.

Esto último se traduce de forma muy negativa en el valor objetivo, pues la función objetivo perjudica notablemente que un turno no tenga la cantidad mínima de empleados.

Además, el algoritmo dinámico está diseñado para obtener una solución óptima para el problema que resuelve, que en este caso consiste únicamente en la asignación mínima de empleados a los turnos de un único día. Es decir, optimiza correctamente este subproblema concreto, que no es el mismo que el que realmente queremos resolver, y además no tiene en cuenta que posteriormente las soluciones de cada día serán combinadas y modificadas para cumplir otras restricciones del problema global, como los descansos de los empleados.

Por el contrario, el algoritmo voraz, al asignar también empleados adicionales cuando estos aportan satisfacción positiva, genera soluciones más completas que resultan más fáciles de adaptar durante el proceso de combinación y reparación de restricciones. Esto provoca que, aunque su decisión no sea óptima a nivel local, el resultado final obtenido tras aplicar las fases posteriores del algoritmo sea, en la práctica, de mayor calidad que el obtenido mediante el algoritmo dinámico.

Conclusiones

En conclusión, el algoritmo dinámico podría parecer, a priori, una mejor alternativa, ya que los algoritmos de programación dinámica suelen garantizar soluciones óptimas para el problema que resuelven, debido al principio de optimalidad. Sin embargo, en este caso la calidad de las soluciones obtenidas es inferior a la del algoritmo voraz.

Esto se debe a que el algoritmo dinámico implementado no modela exactamente el problema original de planificación de empleados, sino una versión simplificada basada en la asignación de slots correspondientes a los mínimos de empleados por turno. Como consecuencia, optimiza únicamente la selección de empleados para cubrir dichos mínimos, sin considerar todas las posibles combinaciones de asignaciones que podrían maximizar la satisfacción global, ni la asignación de empleados extra para que en la fase de combinación no se perjudique tanto el valor objetivo obtenido.

Por este motivo, la solución obtenida puede no ser óptima respecto al objetivo real del problema, permitiendo que el algoritmo voraz, que selecciona directamente los empleados con mayor satisfacción disponible para cada turno, obtenga valores objetivo superiores.

Además, el algoritmo dinámico presenta peores tiempos de ejecución, debido a las operaciones adicionales que requiere la programación dinámica: la construcción de la tabla con todas las soluciones parciales, el proceso de reconstrucción de la solución final y, en este caso concreto, la traducción entre slots y turnos. Por último, mencionar que también hace uso de más recursos de memoria, al hacer uso de más estructuras de datos auxiliares.

Extensión

Se ha intentado implementar un algoritmo para instancias de un día que mejore el resultado de los anteriores. Esto ha sido una tarea bastante complicada, pues el enfoque de doble pasada y maximización de la satisfacción en el algoritmo voraz era una muy buena aproximación.

Para lograr esto se ha optado por implementar un algoritmo basado en intercambios, el cual realiza las dos pasadas características del algoritmo voraz implementado, y tras esto realiza una fase de intercambios donde, para cada par de turnos y empleados que trabajan en ellos posible, intercambia a los empleados si esto mejora la satisfacción.

No obstante, este algoritmo añade mucha complejidad adicional, y mejora sólo muy levemente las soluciones, incluso para algunas instancias las empeora. Esto se debe a que el enfoque de priorizar que los empleados mínimos tengan la máxima satisfacción posible en el algoritmo voraz, antes de añadir empleados con mejor satisfacción adicionales, hacía que al eliminar turnos trabajados en la fase de combinación, como esta también tiene un enfoque de dos pasadas y prioriza eliminar trabajos que no rompen los empleados mínimos por turnos, los turnos trabajados que se eliminaran estuvieran “bien” distribuidos.

Sin embargo, al realizar los intercambios, si hay algún turno que tenga satisfacciones peores que los demás para los empleados, esto causará que este turno pase a tener todas las peores satisfacciones, y, en la fase de combinación, se eliminarán primero los empleados de este turno, conllevando a que sea mucho más común que este tipo de turnos se quede sin la cantidad mínima de empleados eventualmente.

Ejemplo de resultados obtenidos (arriba el nuevo algoritmo y abajo el voraz):

```
o: Tiempo de ejecución: 0.0092076 segundos, Valor objetivo: 644.4
Tiempo de ejecución: 0.00305574 segundos, Valor objetivo: 638.6

10, Empleados: 30
o: Tiempo de ejecución: 0.0144408 segundos, Valor objetivo: 1217.6
Tiempo de ejecución: 0.00607245 segundos, Valor objetivo: 1141.8
```

Para instancias más grandes:

```
: 10, Empleados: 30
o: Tiempo de ejecución: 0.288516 segundos, Valor objetivo: 16605.2
Tiempo de ejecución: 0.119924 segundos, Valor objetivo: 17656.4

: 10, Empleados: 30
o: Tiempo de ejecución: 0.35094 segundos, Valor objetivo: 22706.8
Tiempo de ejecución: 0.149473 segundos, Valor objetivo: 23547.6
```